

Python Advanced

DI Leonard Guelmino
20.-22.10.2026



Mühlehner & Tavolato GmbH

Herzlich Willkommen



DI Leonard Guelmino

- HTL Hollabrunn
- BSc & MSc Informatik @TUWien
- Product Owner bei einem Finanzunternehmen
- sofort-wohnen.at & birddoc.eu
- Trainer @WIFI Wien & @MiT
- CTO @mind-the-brain.com
- Ehrenamt @Rotes Kreuz NÖ

Check-In

- whoami?
- Was ist meine aktuelle Rolle im Unternehmen?
- Welches Vorwissen nehme ich in diesen Kurs mit?
- Was erwarte ich mir von diesem Kurs?

Kurs Inhalt

- **Tag 1:**
  - Klassen und Objekte
  - Ableitungen
  - DATETIME-Modul
  - With-Kontextmanager
  - Exceptions
- **Tag 2:**
  - Iteratoren
  - Higher-Order-Functions
  - Generatoren
  - Dekoratoren
  - Regular Expressions
  - KI in der Software Entwicklung
- **Tag 3:**
  - Type Annotations
  - DocString
  - Requests
  - Datenanalyse Pandas
  - Logging
  - Struktur großer Projekte
  - Unittest
  - Datenbank

Kursformat

- Theorie → Übung → Wiederholung → (Pause)
- Fragen jederzeit einwerfen
- Pausen einfordern
- Übungen
  - Hauptteil und Bonusaufgabe
  - Musterlösung wird am Ende geteilt und besprochen

Lab 0

Überprüfung der IDE & Python-Umgebung

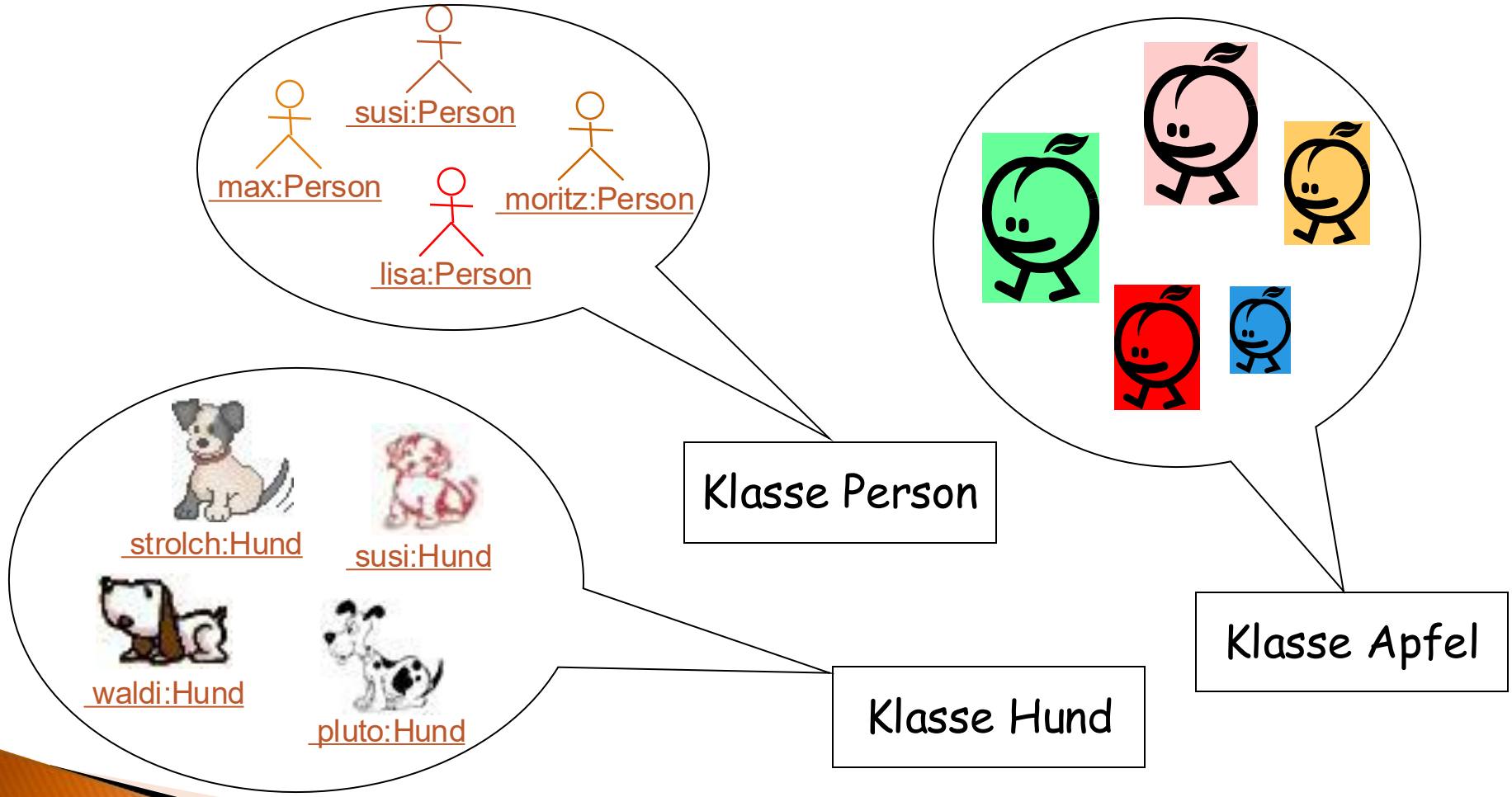
Demo

PyCharm & Debugging

Objektorientierte Konzepte

- Klassen, Objekte, Membership

Klassen und Objekte



Klassen und Objekte

Klasse (Class)



Typ

- Datentyp
- Enthält Attribute und Methoden
- Schablone für Objekte

Objekt (Object)



Wert

- Exemplar ("Instanz") einer Klasse ("Wert")

Klassen und Objekte

Klasse



str (String)

Objekt



"Herr Nilsson"

Klassen und Objekte

Klasse



Person

Objekt



Name: Susi
Alter: 12
Hobby: Reiten

Klassen und Objekte

Klasse



Hund

Objekt



Name: Struppi

MarkenNr: W4711

Klassen und Objekte

- Eine **Klasse** beschreibt eine Menge von Objekten mit gleicher Struktur (Attributen), gleichem Verhalten (Methoden)
- Die **Attribute** (Felder) einer Klasse definieren den Status ihrer Objekte
- Die **Methoden** einer Klasse definieren die Fähigkeiten ihrer Objekte
- Jede **Klasse** hat einen Namen
- Jede **Klasse** muss ausführlich dokumentiert werden

Klassen und Objekte

Es soll eine Klasse erstellt werden, welche eine Uhrzeit als Stunde und Minute kapselt.
Die Werte für Stunde und Minute müssen gesetzt werden können.
Eine beliebige Anzahl von Minuten soll dazugezählt werden können

Klassendefinition

```
class Uhrzeit:
  stunde = None

  # Konstruktor
  def __init__(self):
    self.stunde = 0

  # eine Uhrzeit setzen
  def zeit_setzen(self, stunde, minute=2):
    # Attribute entstehen bei der Zuweisung
    self.stunde = stunde
    self.minute = minute

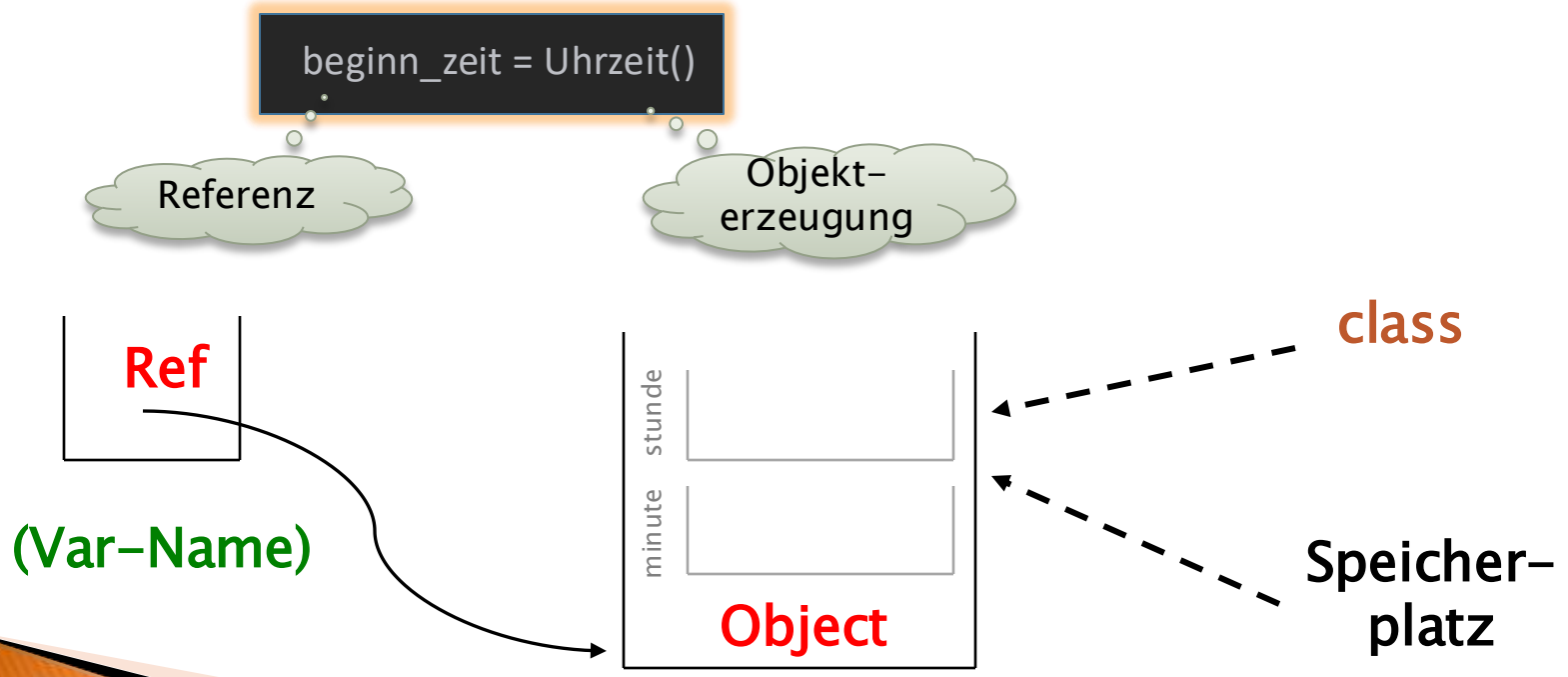
  # die Uhrzeit anzeigen
  def zeit_anzeigen(self):
    print(f"{self.stunde}:{self.minute}")

  # Minuten dazuzählen
  def minuten_dazu(self, minuten):
    self.minute += minuten
```

Instanziierung von Objekten

- Erzeugung von Objekten (Instanziierung) erfolgt in Python ausschließlich dynamisch

- Weitere Verwendung immer über Referenz

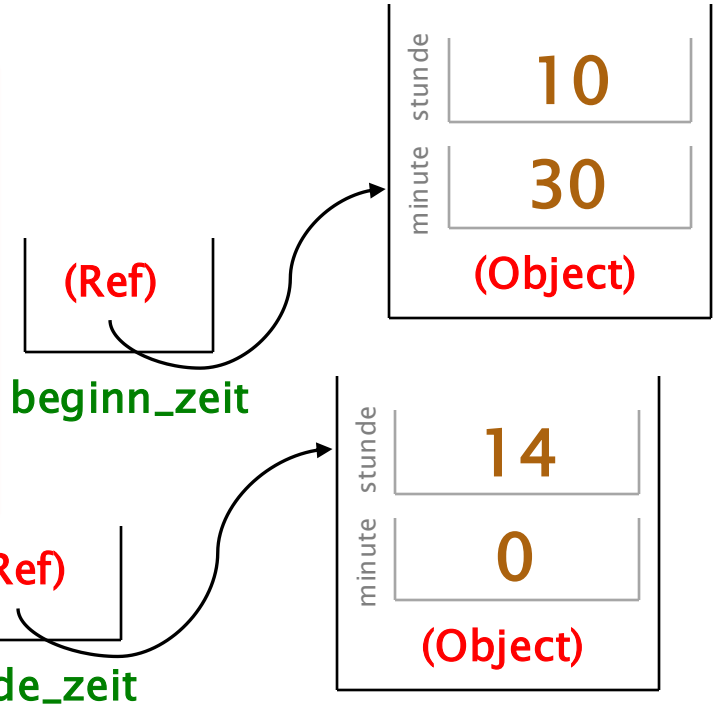


Verwenden von Objekten

- Aufrufen der Methoden
  - erfolgt immer über eine Referenz
  - mit dem Operator für den Memberzugriff

```
beginn_zeit = Uhrzeit()
beginn_zeit.zeit_setzen(10, 30)
beginn_zeit.zeit_anzeigen()


ende_zeit = Uhrzeit()
ende_zeit.zeit_setzen(14, 0)
```



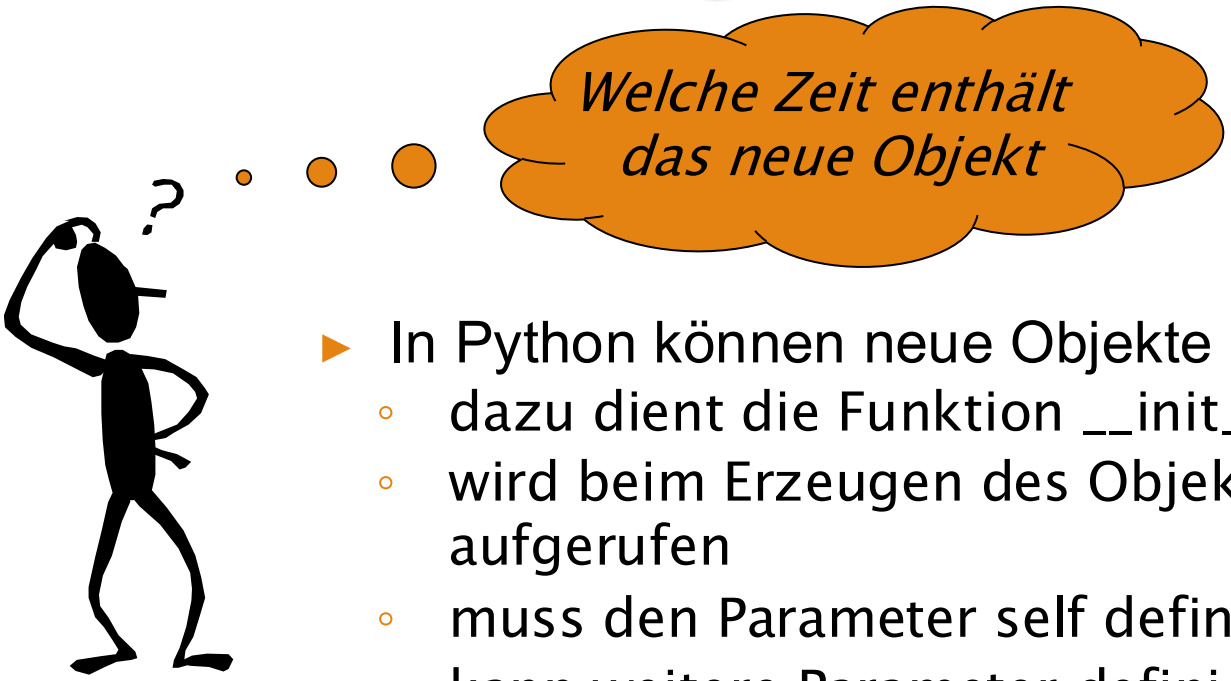
Zugriffsattribute

Datenkapselung ist ein Grundkonzept der OOP

- In Python nur über Konvention/Name Mangling gelöst (nicht sicher)

Zugriff	Syntax	Wer soll zugreifen	Umsetzung
private	*__attribute*	nur die Klasse selbst	Name Mangling
protected	*_attribute*	die Klasse und abgeleitete Klassen	Konvention
public	*attribute*	Jeder, Default in der Klasse	

Initialisierung



- In Python können neue Objekte initialisiert werden
  - dazu dient die Funktion __init__(self)
  - wird beim Erzeugen des Objekts automatisch aufgerufen
  - muss den Parameter self definieren
  - kann weitere Parameter definieren
  - kann Defaultparameter (Standardwerte) verwenden
  - darf keinen Wert zurückliefern (return)

Initialisierung

```
class Uhrzeit:

 # Initialisierungsfunktion mit Parametern
  def __init__(self, stunde=0, minute=0):
    self.__stunde = stunde  # Definition als "private" Attribut
    self.__minute = minute
```

```
ende_zeit = Uhrzeit(14, 0)
```

```
beginn = Uhrzeit()
beginn.zeit_setzen(10, 30)
```

Operator-Overloading

In Python-Klassen können die normalen Operatoren mit Klassen-Methoden überladen werden. Damit sind Operationen zwischen Klassen möglich.

```
class A:
  def __init__(self, a):
    self.a = a

  # adding two objects
  def __add__(self, o):
    return self.a + o.a


ob1 = A(1)
ob2 = A(2)
print(ob1 + ob2)
```

Operator-Overloading

+	__add__(self, other)
-	__sub__(self, other)
*	__mul__(self, other)
/	__truediv__(self, other)
//	__floordiv__(self, other)
%	__mod__(self, other)
**	__pow__(self, other)
>>	__rshift__(self, other)
<<	__lshift__(self, other)
&	__and__(self, other)
|	__or__(self, other)
^	__xor__(self, other)

-=	__ISUB__(SELF, OTHER)
+=	__IADD__(SELF, OTHER)
*=	__IMUL__(SELF, OTHER)
/=	__IDIV__(SELF, OTHER)
//=	__IFLOORDIV__(SELF, OTHER)
%=	__IMOD__(SELF, OTHER)
**=	__IPOW__(SELF, OTHER)
>>=	__IRSHIFT__(SELF, OTHER)
<<=	__ILSHIFT__(SELF, OTHER)
&=	__IAND__(SELF, OTHER)
|=	__IOR__(SELF, OTHER)
^=	__IXOR__(SELF, OTHER)

<	__LT__(SELF, OTHER)
>	__GT__(SELF, OTHER)
<=	__LE__(SELF, OTHER)
>=	__GE__(SELF, OTHER)
==	__EQ__(SELF, OTHER)
!=	__NE__(SELF, OTHER)

Instanz- vs Klassen-Attribute

	Instanzmember	Statische Member
Zugehörigkeit	sind an ein Objekt gebunden	gehören direkt zur Klasse
Verwendung	es muss zuvor ein Objekt erzeugt und initialisiert worden sein	es ist kein Objekt erforderlich
Zugriff	nur über eine Referenz/Variable	ohne Referenz, über den Klassennamen
Definition	in einer Methode bzw. __init__()	direkt im Klassenblock
Einsatz	○ mehrere Exemplare der Klasse mit jeweils eigenen Werten; ○ fortgeschrittene OOP-Techniken nutzen (z.B. Vererbung)	○ Utility-Methoden, die ohne vorherige Instanziierung verwendbar sein sollen; ○ "Globale" Methoden

Instanz-Member

Instanz-Attribut

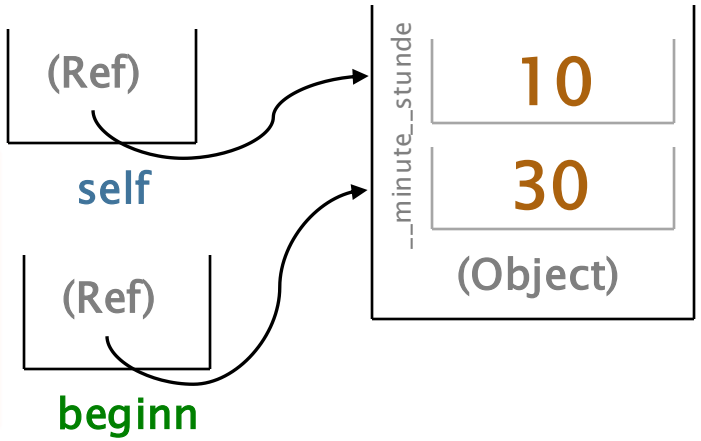
- "Gewöhnliches" Attribut: existiert 1x pro Objekt

Instanz-Methode

- kann auch die Instanz-Attribute lesen und ändern
- dazu wird beim Aufruf die Referenz auf das Objekt implizit mitgegeben: self

```
beginn = Uhrzeit()
beginn.zeit_setzen(10, 30)
```

```
def zeit_setzen(self, stunde, minute):
    # Attribute entstehen bei der Zuweisung
    self.__stunde = stunde
    self.__minute = minute
```



Statische Member

- Definition direkt im Block der Klasse
- Statische **Attribute** (Klassen-Attribute)
  - existieren genau 1x für die Klasse
  - existieren unabhängig von Instanzen der Klasse
- Statische **Methoden** (Klassen-Methoden)
  - können diese Attribute und andere statische Methoden verwenden
  - können keine Instanzmember verwenden (kein self)
- Zugriff von außen
  - erfolgt über den Klassennamen

Statische Member

Das Klassenattribut "__object_count" gibt es nur einmal für die ganze Klasse!

Das Instanzattribut "__id" existiert 1x pro Objekt

```
class CountClass:
  __object_count = 0

  def __init__(self, id):
    self.__id = id
    CountClass.__object_count += 1

  def get_id(self):
    return self.__id

  def get_object_count(self):
    return CountClass.__object_count
```

In Klassenmethoden kann nur auf Klassenattribute zugegriffen werden

```
c1 = CountClass(5)
c2 = CountClass(7)

id1 = c1.get_id() # == 5
id2 = c2.get_id() # == 7

# == 2
count =
CountClass.get_object_count(c2)
```

Der Zugriff von außen erfolgt über den Klassennamen

Lab 1

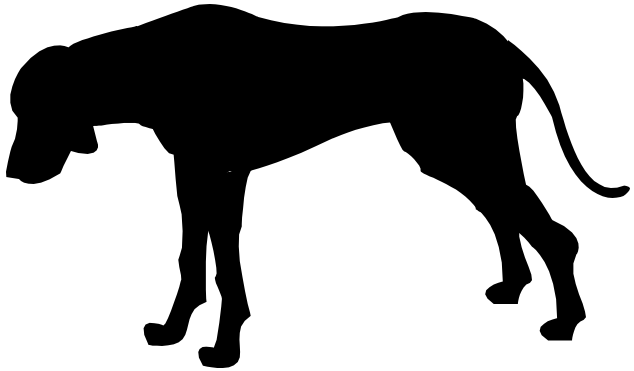
Klassen & Objekte – Das Bankkonto

Vererbung

- Spezialisierung von Klassen

Vererbung – Ableitung

- Die abgeleitete Klasse ist eine Spezialisierung einer (Basis-)Klasse



Dackel ist abgeleitet von Hund



Hund

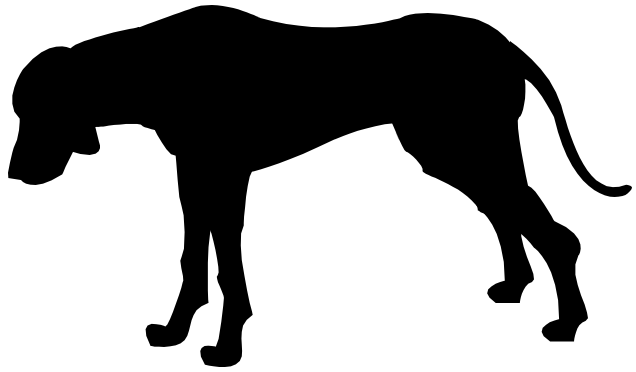
- Basisklasse
  - Grundattribute und -methoden von Hunden

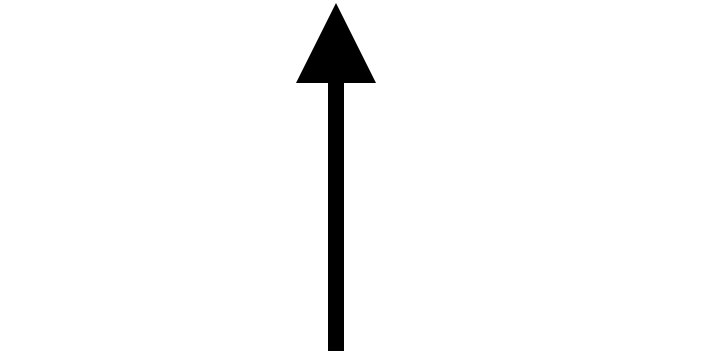
Dackel

- Wie Hund, aber:
  - einige Dinge anders
  - zusätzliche Funktionalität

Vererbung – Ableitung

class **Hund**





class **Dackel(Hund)**



Dackel wird von Hund abgeleitet

Vererbung

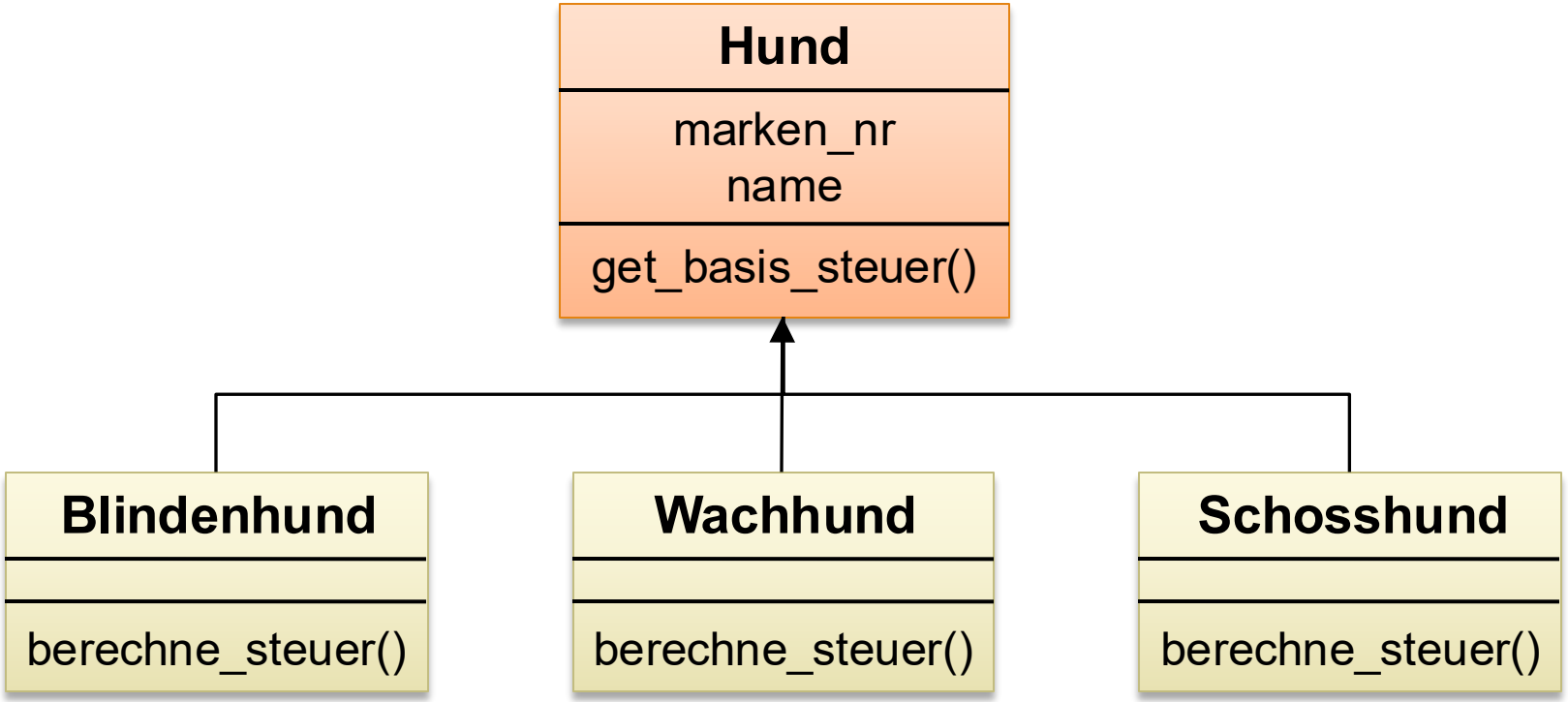
- Beispiel "Hundesteuer berechnen"

Die Hundesteuer beträgt 100 EUR.
Wachhunde zahlen nur die Hälfte.
Und Blindenhunde sind überhaupt von der Steuer befreit.

Berechnet werden soll die Steuer für Blindenhunde, Wachhunde und Schoßhunde

Vererbung

- Beispiel "Hundesteuer berechnen"



Blindenhund, Wachhund und Schoßhund sind Spezialisierungen von Hund, mit jeweils eigener Berechnungsmethode

Vererbung

```python
class Hund:
  def __init__(self, name, marken_nr):
    self.name = name
    self.marken_nr = marken_nr

  # Normale Steuer berechnen
  def get_basis_steuer(self):
    # "Normale" Steuer
    return 100

  def get_name(self):
    return self.name

  def get_marken_nr(self):
    return self.marken_nr
```

```python
class Wachhund(Hund):
  def __init__(self, einsatz_ort, name, marken_nr):
    super().__init__(name, marken_nr)
    self.einsatz_ort = einsatz_ort

  # Hundesteuer für Wachhunde
  def berechne_steuer(self):
    # zahlen nur die Hälfte
    return self.get_basis_steuer() / 2

  def get_einsatz_ort(self):
    return self.einsatz_ort
```





Vererbung – Initialisierung

- Beim Erzeugen eines Objekts einer abgeleiteten Kasse wird die Initialisierung der Basisklasse NICHT automatisch aufgerufen
  - kann/soll in der Initialisierung der abgeleiteten Klasse aufgerufen werden
  - erst dann existieren die Attribute!

```
class Hund:
  def __init__(self):
```

```
class Wachhund(Hund):
    def __init__(self):
      Hund.__init__(self)
       # oder
       super().__init__()
```

```
rex = Wachhund()
```



1. Initialisierung von Hund
2. Initialisierung von Wachhund

Vererbung: super-Aufruf

- Falls die Basisklassen-Initialisierung Parameter hat
  - werden die Parameter beim Aufruf an die __init__() Methode der Basisklasse übergeben

```
class Hund:
  def __init__(self, name, marken_nr):
    self.name = name
    self.marken_nr = marken_nr
```

```
class Wachhund(Hund):
  def __init__(self, einsatz_ort, name, marken_nr):
    super().__init__(name, marken_nr)
    self.einsatz_ort = einsatz_ort
```

```
rex = Wachhund('Wien', 'Rex', 100)
```

Lab 2

Vererbung – Spezialisierte Konten

Datum und Zeit

- Modul datetime

datetime

- Klassen die ein einfaches Datum (Tag, Monat, Jahr) bzw. eine einfache Zeit speichern
  - Aktueller Wert: today() für date
  - Initialisierungen mit Parametern
  - Zugriff auf die einzelnen Komponenten: hour, minute, second bzw. day, month, year etc.

```
import datetime  # benötigtes Modul

heute = datetime.date.today()
print("Heute ist der", heute)  # → "Heute ist der 2026-10-20"

zeit = datetime.time(11, 36, 23)
print("Jetzt ist es", zeit) # → "Jetzt ist es 11:36:23"
print("Stunden =", zeit.hour) # → "Stunden = 11"
```

datetime

- Klasse für einen kompletten Zeitpunkt (Datum und Zeit)
  - Aktueller Wert: now() für date
  - Initialisierungen mit Parametern
  - Zugriff auf die einzelnen Komponenten: hour, minute, second bzw. day, month, year etc.

```
import datetime  # benötigtes Modul

jetzt = datetime.datetime.now()
print("Aktuell", jetzt)         # → "Aktuell 2026-10-20 10:48:19"
print("Stunden =", jetzt.hour) # → "Stunden = 10"
print("Jahr =", jetzt.year)    # → "Jahr = 2026"
```

Umwandlung datetime – String

- Unterstützt das Standard ISO Format: 2026-04-18T06:35:13.546
- Erzeugen eines ISO String .isoformat()
- Aus ISO String parsen datetime.fromisoformat(string)
- Einen beliebig formatierten String erzeugen .strftime(format)
- Aus einem beliebig formatierten String parsen datetime.strptime(string, format)

Kontext Manager: with

- mit Python

Context Manager: with

- `with` erzeugt einen Block von Statements.
  - Vor der ersten Ausführung des Blockes erfolgt eine Initialisierung (Aufruf `__enter__()`),
  - nach Beendigung des Blockes erfolgen Aufräumarbeiten (Aufruf `__exit__()`).
- Das `with`-Statement kann statt einem `try…finally` Block verwendet werden (weniger Code notwendig)
- Verwendung:
  - Lock/Unlock von Resourcen
  - Open/Close von Resourcen
  - Datenbank Commitment/Rollback
  - Zeitmessung von Code

Context Manager: with

```
# ohne Context Manager 'with'
file = open('file_path', 'w')
try:
  file.write('hello world')
finally:
  file.close()



# mit Context Manager 'with'
with open('file_path', 'w') as file:
  file.write('hello world !')
```

So funnktioniert open()

```
class FileManager():
  def __init__(self, filename, mode):
    self.filename = filename
    self.mode = mode
    self.file = None

  def __enter__(self):
    self.file = open(self.filename,
    self.mode)
    return self.file

  def __exit__(self, exex_type, exec_val, exc_tb):
    self.file.close()
```

Lab 3

Datetime & With - Transaktions-Logging

Exceptions

- In Python

Exceptions

- `try`: Umschließt den "riskanten" Code, der eine Exception auslösen könnte.
- `except`: Behandelt spezifische Fehler, die im try-Block auftreten.
- `else`: Wird nur ausgeführt, wenn der try-Block keine Exception ausgelöst hat. Fängt except-Zweige darüber **nicht** ab.
- `finally`: Wird immer ausgeführt, egal ob eine Exception aufgetreten ist oder nicht. Ideal für Aufräumarbeiten (z.B. Schließen von Ressourcen).

Exceptions

```python
def get_resource_content(resource_manager, resource_id):
    """
    Ein robustes Beispiel, das alle Blöcke nutzt.
    """
    resource = None
    try:
        # 1. try: Code, der fehlschlagen kann
        resource = resource_manager.acquire(resource_id)
        data = resource.read_data()

    except ResourceNotFoundError as e:
        # 2. except: Spezifische Behandlung
        logging.warning(f"Ressource {resource_id} nicht gefunden: {e}")
        return None  # Fehler behandelt, Abbruch mit None

    except ReadTimeoutError as e:
        # 2. except: Anderer spezifischer Fehler
        logging.error(f"Timeout beim Lesen von Ressource {resource_id}", exc_info=True)
        raise  # Fehler kann hier nicht behandelt werden -> weiterleiten

    else:
        # 3. else: Nur wenn try ERFOLGREICH war
        logging.info(f"Ressource {resource_id} erfolgreich gelesen.")
        return data

    finally:
        # 4. finally: Wird IMMER ausgeführt
        if resource:
            resource_manager.release(resource)
            logging.debug(f"Ressource {resource_id} freigegeben.")
```

Exceptions in großen Projekten

In großen Anwendungen reicht das reine Abfangen von Fehlern nicht aus. Es geht um **Isolation**, **Logging** und **klare API-Grenzen**.

```
try:
  # ... irgendein Code ...
except Exception as e:
  logging.error("Ein Fehler ist aufgetreten")
```

```
try:
  user = db.get_user(user_id)
except DatabaseDownError:
  # Behandelbar: Retry-Logik oder Fallback
  return get_user_from_cache(user_id)
except UserNotFoundError:
  # Behandelbar: 404-Antwort generieren
  return http_response_404()
```

Exceptions in großen Projekten

Definieren Sie eine Basis-Exception für Ihr Modul und leiten Sie spezifische Fehler davon ab.

```python
class PaymentGatewayError(Exception):
    """Basis-Exception für alle Fehler in diesem Modul."""
    pass

class AuthenticationError(PaymentGatewayError):
    """Falsche API-Keys."""
    pass

class InsufficientFundsError(PaymentGatewayError):
    """Guthaben nicht ausreichend."""
    pass

class NetworkError(PaymentGatewayError):
    """Problem mit der Payment-API-Verbindung."""
    pass
```

```python
def process_payment_view(request):
    try:
        payment_gateway.charge(request.user, request.amount)
    except payment_gateway.InsufficientFundsError:
        return render_error("Zahlung abgelehnt: Guthaben zu gering.")
    except payment_gateway.PaymentGatewayError as e:
        # Fängt alle ANDEREN Payment-Fehler (Auth, Network)
        logging.error(f"Allg. Zahlungsfehler: {e}")
        return render_error("Ein Fehler im Zahlungssystem ist aufgetreten.")
    # Ein TypeError hier wird NICHT gefangen -> das ist ein Bug!
```

Exceptions in großen Projekten

Exception Chaining

- Verwenden Sie `raise ... from ...`, um die ursprüngliche Exception "anzuketten".

```
def charge(user, amount):
  try:
    response = requests.post("https://api.payment.com/charge", data={...})
    response.raise_for_status()  # Löst HTTPError bei 4xx/5xx aus
  except requests.exceptions.Timeout as e:
    # Fehler loggen (WICHTIG!)
    logging.error("Payment API Timeout", exc_info=True)
    # Eigene Exception auslösen UND die Ursache (e) mitgeben
    raise NetworkError("API-Timeout beim Bezahlen") from e
  except requests.exceptions.HTTPError as e:
    if e.response.status_code == 402:
      raise InsufficientFundsError("API meldet 402") from e
    raise AuthenticationError("API-Authentifizierung fehlgeschlagen") from e
```

Exceptions in großen Projekten

`finally` durch Context Manager (`with`) ersetzen

```
# Mühsam und fehleranfällig:
lock = threading.Lock()
lock.acquire()
try:
  do_critical_stuff()
finally:
  lock.release()
```

```
# Robust, lesbar und garantiert das 'release':
lock = threading.Lock()
with lock:
  do_critical_stuff()
# Das Gleiche gilt für Dateien:

with open("data.txt", "r") as f:
  process(f)
  # 'f.close()' wird garantiert aufgerufen, selbst
   wenn 'process(f)' abstürzt.
```

Lab 4

Exceptions – Robuste Transaktionsverarbeitung